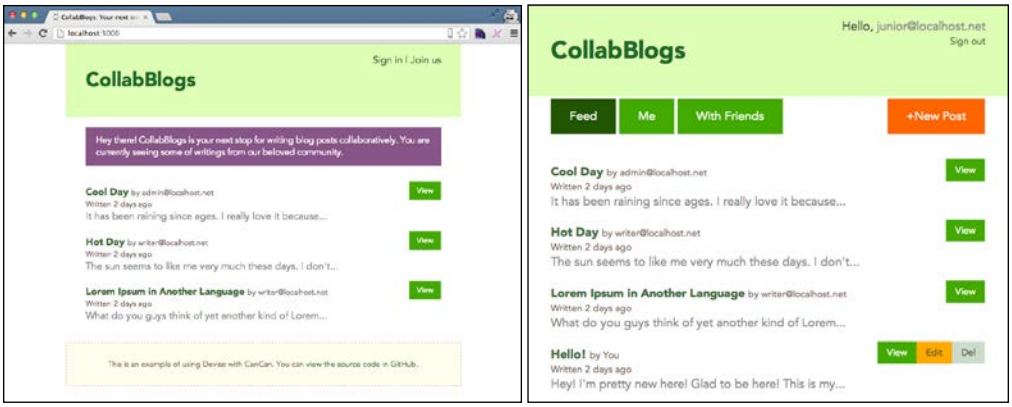
With custom CSS and JavaScript we have in our sample project, we can have a list of posts displayed with actions allowed for the visitors, like the following:



Screenshots of a guest user's page (left) and a logged-in user's page (right)

Advanced CanCan usages

The previous abilities we learned through our CollabBlogs application are enough for us to get started. However, what if our application requires complex authorization rules? We have a few remaining complex rules unimplemented, and the abilities we have applied are far simpler than those complex rules.

After we have plenty of rules, we should try to simplify parts of the authorization process and test the rules' correctness. This is to ensure our application behaves as expected.

In this section, we are going to discuss defining rules using SQL, simplifying authorization checks on controllers, and ensuring abilities' correctness.

Defining rules using SQL

As mentioned before, the `can` and `cannot` methods we defined in `app/models/abilities.rb`, are able to take SQL conditions and block parameters. The SQL conditions parameter is useful for filtering resources that match the ability (using `Model#accessible_by(ability, action)`). The block parameter is very often used for authorizing actions (for example, edit) on a single resource.

www.it-ebooks.info